DELITOS CIBERNÉTICOS

LAB SEMANA 2 — CONCEPTOS DE SEGURIDAD EN REDES

Reporte de avance — Documento de trabajo



Alumno: ElVigilante

Carrera: Ingeniería en Seguridad Informática y Redes

Matrícula: ?

Materia: Delitos Cibernéticos

Nombre del Docente: Dulce Vázquez Cano

Fecha de entrega: 13 de septiembre de 2026

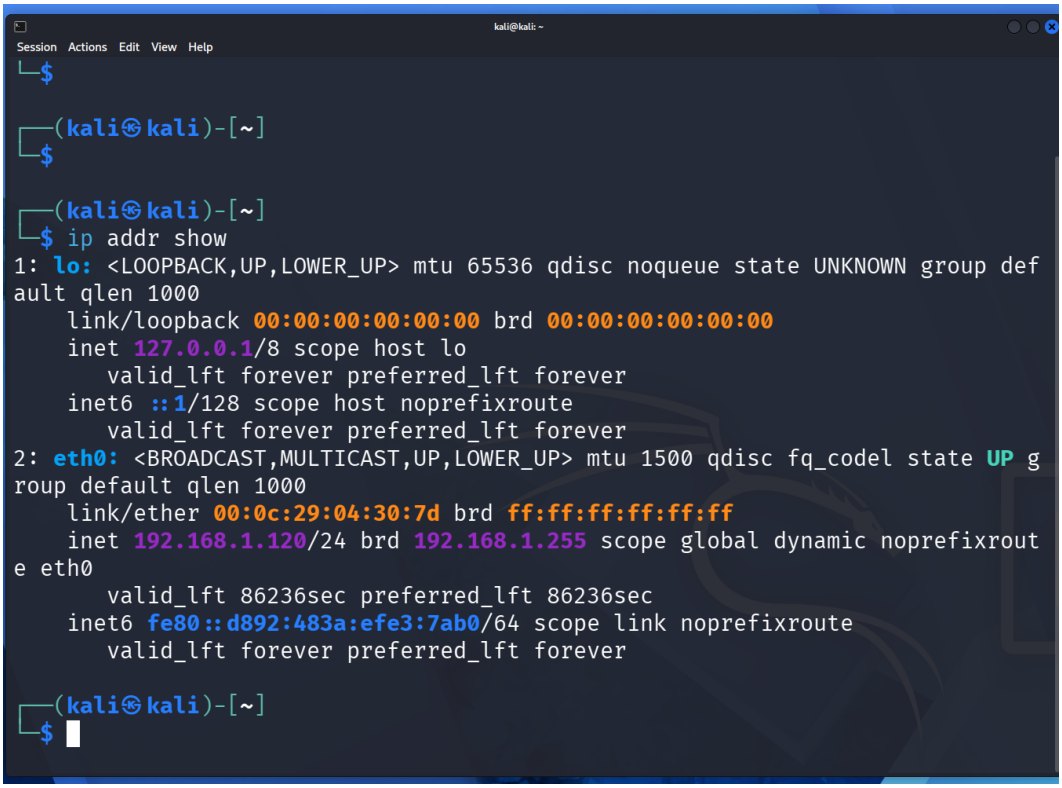
Declaración de conducta ética: las herramientas utilizadas en este laboratorio (nmap, tshark/Wireshark, arpspoof, hping3, openssl) fueron empleadas exclusivamente dentro de un entorno virtual controlado, (VM1 Kali Linux y VM2 Metasploitable2, ambas en red Bridge dentro de la red doméstica del alumno). Su uso contra redes o sistemas de terceros sin autorización expresa constituye un delito conforme al artículo 211 bis del Código Penal Federal.

Entorno de Laboratorio

Host: Windows 10/11, VMware Workstation, red Wi-Fi doméstica. VM1 (Atacante/Analista): Kali Linux 2026.1, modo de red Bridged. VM2 (Víctima): Metasploitable2-Linux, modo de red Bridged. Nota: se utilizó Metasploitable2 en lugar de Ubuntu Server para contar con servicios intencionalmente vulnerables (FTP, IRC, servicios web) que permiten ilustrar con mayor claridad los conceptos de reconocimiento y explotación de la guía, sobre la misma base de red Bridge y los mismos protocolos (ARP, DNS, TCP, HTTP) solicitados.

Dato	VM1 — Kali	VM2 — Metasploitable2
IP	192.168.1.120	192.168.1.121
MAC	00:0c:29:04:30:7d	00:0c:29:e5:5f:a2
Interfaz	eth0	eth0

Gateway de la red (router doméstico): 192.168.1.254, MAC bc:d2:06:d2:f9:28 (Huawei), hostname dev.opt.



Configuración de red confirmada en Kali (ip addr show).

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:e5:5f:a2
          inet addr:192.168.1.121  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fee5:5fa2/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:61 errors:0 dropped:0 overruns:0 frame:0
          TX packets:87 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6628 (6.4 KB)  TX bytes:8950 (8.7 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:100 errors:0 dropped:0 overruns:0 frame:0
          TX packets:100 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:23481 (22.9 KB)  TX bytes:23481 (22.9 KB)

msfadmin@metasploitable:~$
```

Configuración de red confirmada en Metasploitable2 (ifconfig).

```
┌──(kali㉿kali)-[~]
└─$ ping -c 4 192.168.1.121
PING 192.168.1.121 (192.168.1.121) 56(84) bytes of data.
64 bytes from 192.168.1.121: icmp_seq=1 ttl=64 time=0.672 ms
64 bytes from 192.168.1.121: icmp_seq=2 ttl=64 time=0.347 ms
64 bytes from 192.168.1.121: icmp_seq=3 ttl=64 time=0.403 ms
64 bytes from 192.168.1.121: icmp_seq=4 ttl=64 time=0.335 ms

── 192.168.1.121 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.335/0.439/0.672/0.136 ms

┌──(kali㉿kali)-[~]
└─$
```

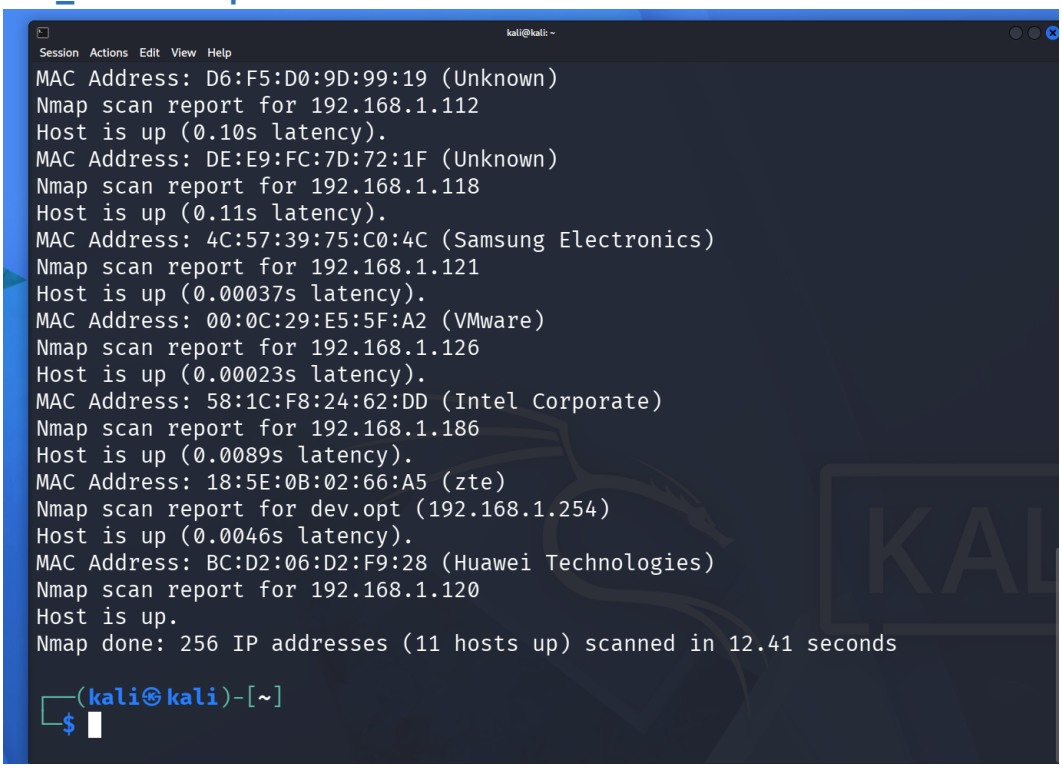
Prueba de conectividad entre VM1 y VM2 (ping).

Práctica 1 — Reconocimiento de Red y Escaneo de Puertos

Fase 1A — Descubrimiento de Hosts con ARP e ICMP

Se identificó la subred de trabajo con ip addr show (192.168.1.0/24) y se ejecutó un ping sweep con nmap -sn sobre la subred completa, descubriendo 11 hosts activos, incluyendo routers, dispositivos móviles y ambas VMs del laboratorio. Nmap identificó el fabricante de cada dispositivo a partir del OUI (primeros 3 bytes) de su dirección MAC, sin necesidad de acceder a ningún puerto ni servicio — pura información de capa 2 expuesta por diseño del protocolo ARP/Ethernet.

Evidencia P1_E1 — nmap -sn



P1_E1 — nmap -sn 192.168.1.0/24: descubrimiento de 11 hosts activos.

Tras el escaneo, la tabla ARP local (arp -a) únicamente retuvo las entradas de los hosts con comunicación reciente (Metasploitable2 y el gateway), evidenciando el comportamiento de caducidad automática de las entradas ARP — un mecanismo central para entender el ataque de ARP Spoofing desarrollado en la Práctica 2.

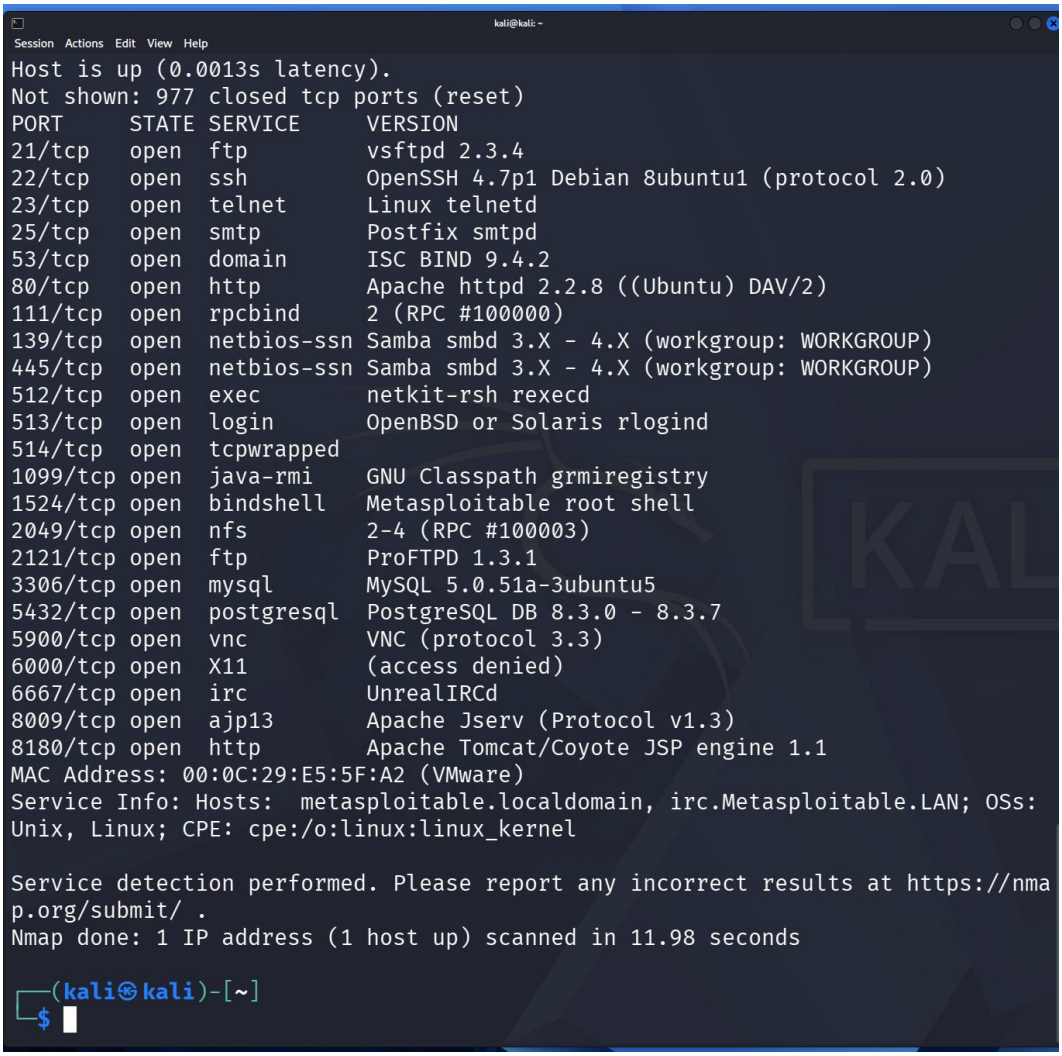


arp -a: solo persisten las entradas con comunicación activa reciente.

Fase 1B — Escaneo de Puertos con Nmap

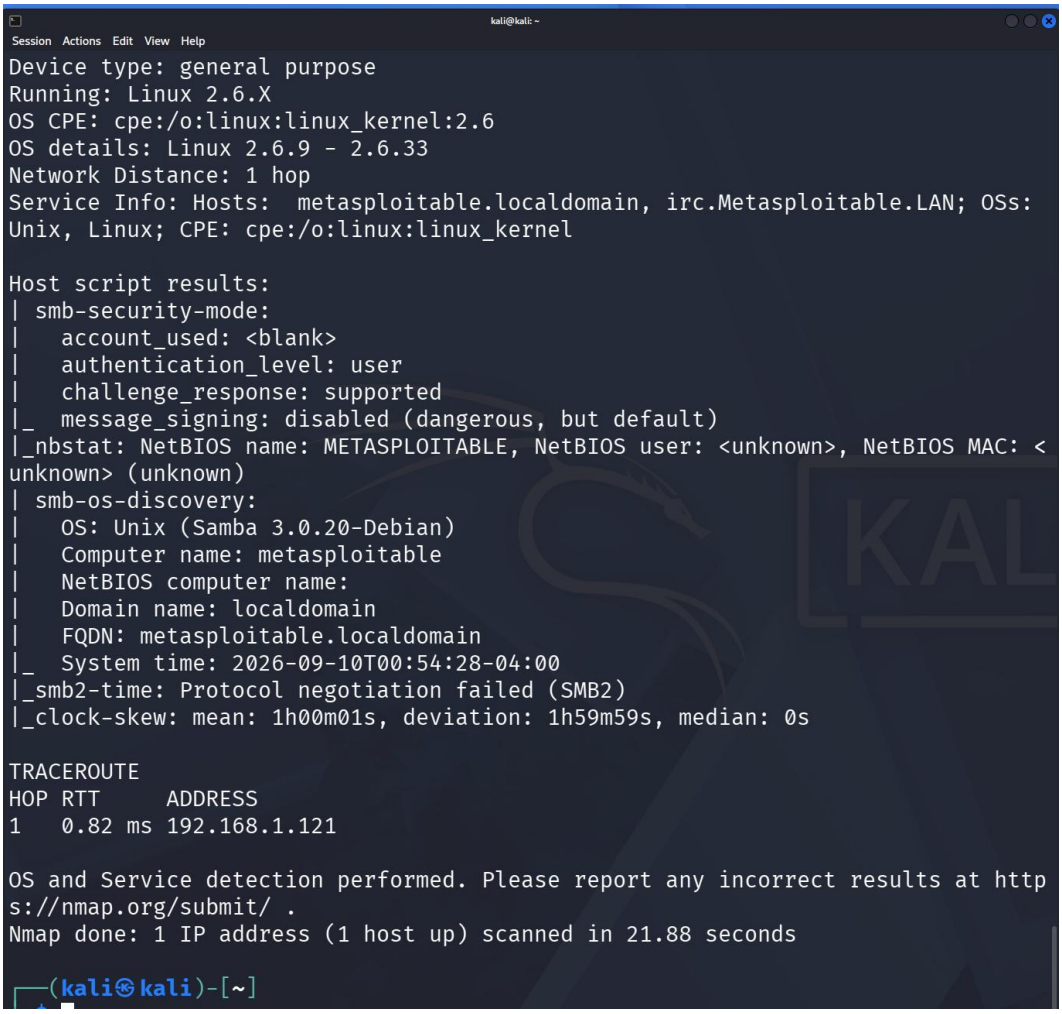
Contra la IP específica de Metasploitable2 (192.168.1.121) se identificaron 23 puertos TCP abiertos con sus versiones de servicio mediante nmap -sV. Destacan tres hallazgos críticos: (1) vsftpd 2.3.4 en el puerto 21, versión con una puerta trasera intencional documentada como CVE-2011-2523; (2) UnrealIRCd en el puerto 6667, también con una puerta trasera de cadena de suministro documentada (CVE-2010-2075); y (3) el puerto 1524, que nmap identificó directamente como "Metasploitable root shell", una shell de administrador sin ningún control de acceso.

Evidencia P1_E2 — nmap -sV



P1_E2 — nmap -sV 192.168.1.121: 23 puertos abiertos identificados.

De forma complementaria, un escaneo agresivo (nmap -A) confirmó el sistema operativo como Linux 2.6.9–2.6.33, y reveló mediante el script smb-os-discovery información de configuración (nombre de equipo, dominio, FQDN, hora del sistema) sin necesidad de autenticación. También se detectó que la firma de mensajes SMB está deshabilitada ("message_signing: disabled (dangerous, but default)"), condición que habilita ataques de tipo SMB relay.

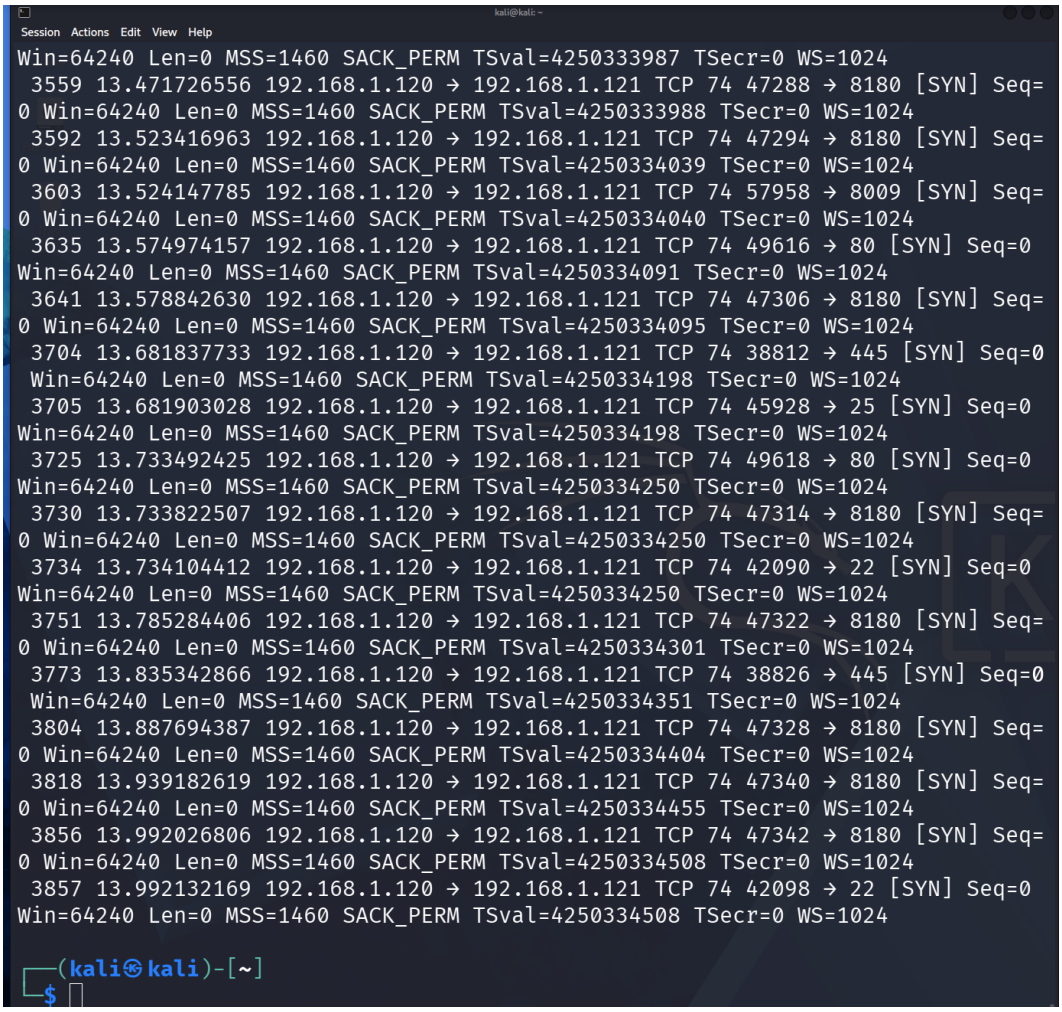


nmap -A (evidencia complementaria): detección de sistema operativo y hallazgos SMB.

Fase 1C — Captura de Paquetes SYN durante el Escaneo

Utilizando tshark en modo texto (equivalente funcional a la interfaz gráfica de Wireshark, dado que las VMs de este entorno no cuentan con integración de mouse) se capturó el tráfico generado por nmap -A -T4, filtrando exclusivamente los paquetes con la bandera SYN activa y ACK inactiva (tcp.flags.syn==1 && tcp.flags.ack==0). El resultado muestra al menos 16 intentos de conexión visibles en la captura (posiblemente más, dado el scroll limitado de la terminal) desde el puerto origen de Kali hacia distintos puertos destino de Metasploitable2 (8180, 8009, 80, 445, 25, 22, entre otros), evidenciando la huella de red característica de un escaneo de puertos: múltiples intentos de conexión de tres vías iniciados y nunca completados por el lado del escáner una vez identificado el estado del puerto. Esto es exactamente la "huella digital" que los sistemas IDS/IPS están diseñados para detectar.

Evidencia P1_E3 — Captura de paquetes SYN



P1_E3 — Paquetes SYN capturados durante el escaneo nmap -A.

Reflexión Requerida — Práctica 1

Evidencia P1_E4 — Reflexión escrita

¿Qué es un ataque de reconocimiento y por qué Nmap es legal usarlo en tu propia red pero no en redes ajenas?

Un ataque de reconocimiento es la fase inicial de cualquier intrusión, donde el atacante recolecta información sobre el objetivo —hosts activos, puertos abiertos, servicios, versiones de software y sistema operativo— sin necesariamente explotar nada todavía. En este laboratorio lo comprobé de forma práctica: con nmap -sn 192.168.1.0/24 descubrí 11 dispositivos activos en mi red doméstica identificando su fabricante por la dirección MAC, y con nmap -sV y nmap -A contra mi VM víctima (Metasploitable2, 192.168.1.121) obtuve 23 puertos abiertos con sus versiones exactas de software, además del sistema operativo y detalles de

configuración SMB, todo sin necesitar credenciales ni autorización explícita de la máquina objetivo.

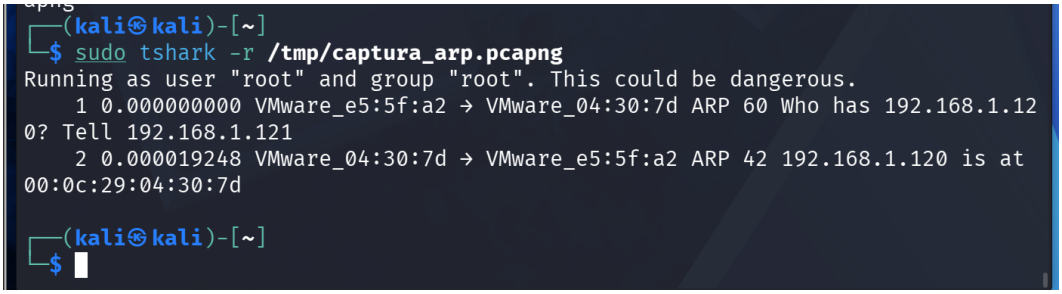
Nmap en sí mismo no es una herramienta maliciosa: es un escáner de red legítimo, ampliamente usado en administración de sistemas y auditorías de seguridad autorizadas. La legalidad de su uso no depende de la herramienta sino de la autorización sobre el sistema objetivo. En mi propia red doméstica y mi propia VM soy el dueño y administrador legítimo de esos equipos, por lo que escanearlos no constituye ningún delito. En cambio, el artículo 211 bis del Código Penal Federal —citado explícitamente en la cláusula de conducta ética de este laboratorio— sanciona el acceso ilícito a sistemas informáticos ajenos sin autorización; usar exactamente el mismo comando nmap -A contra la red de un tercero, sin su consentimiento expreso, encajaría en ese tipo penal, independientemente de que el escaneo en sí no dañe nada.

Práctica 2 — Captura y Análisis de Tráfico: ARP, DNS y TCP

Fase 2A — Observando el Protocolo ARP

Tras vaciar la caché ARP local (sudo ip -s neigh flush all) y emitir un ping hacia Metasploitable2, se capturó el intercambio ARP correspondiente con tshark (filtro arp). De forma interesante, la solicitud capturada fue emitida por Metasploitable2 preguntando por la MAC de Kali, y no al revés — esto se debe a que únicamente se vació la caché de Kali; Metasploitable2, al no tener en su propia caché la MAC de Kali, tuvo que resolverla para poder responder el eco ICMP. Esto evidencia que ARP es un mecanismo bidireccional e independiente en cada extremo de la comunicación.

Evidencia P2_E1 — ARP Request/Reply con MACs visibles

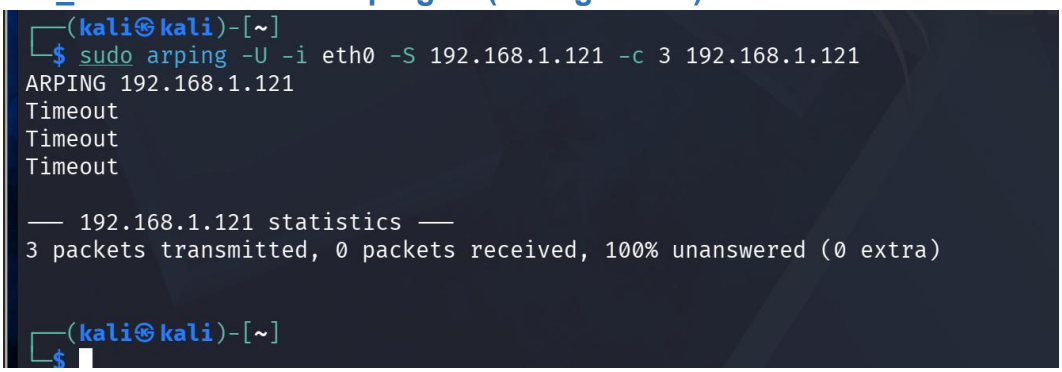


P2_E1 — Intercambio ARP Request/Reply capturado con tshark.

Fase 2B — ARP Gratuito y Ataque de ARP Spoofing

Se ejecutó el comando de ARP gratuito solicitado por la guía usando arping (versión Thomas Habets 2.28, presente por defecto en esta imagen de Kali, con sintaxis distinta a la de iputils-arping referenciada originalmente): sudo arping -U -i eth0 -S 192.168.1.121 -c 3 192.168.1.121. Este comando anuncia de forma no solicitada ("gratuita") que la IP 192.168.1.121 (la de Metasploitable2) está asociada a la MAC de Kali. Los 3 paquetes se transmitieron exitosamente (0% de pérdida en el envío); el "Timeout" reportado por la herramienta es el comportamiento esperado en modo -U, ya que un anuncio gratuito no espera ni requiere respuesta — no es una petición ARP tradicional.

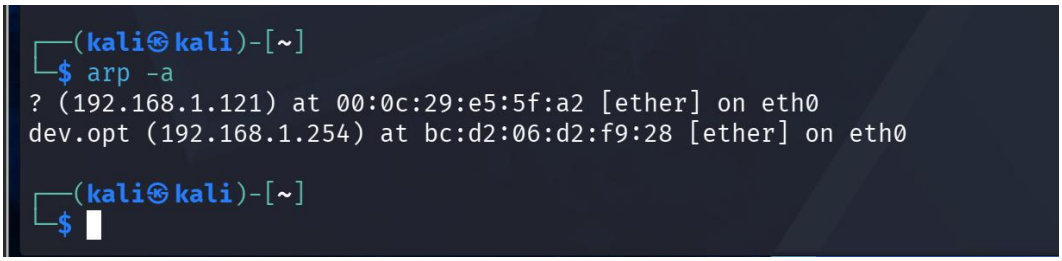
Evidencia P2_E2a — Comando arping -U (ARP gratuito)



P2_E2a — arping -U: 3 anuncios ARP gratuitos transmitidos hacia 192.168.1.121.

Al verificar arp -a en la propia Kali después del comando, la entrada de 192.168.1.121 se mantuvo sin cambios (asociada a su MAC real, 00:0c:29:e5:5f:a2). Este resultado es correcto y esperado: un host no actualiza su propia caché ARP a partir de los anuncios que él mismo origina — el ARP gratuito está diseñado para actualizar la caché de los demás equipos de la red que reciben el anuncio, no la del emisor.

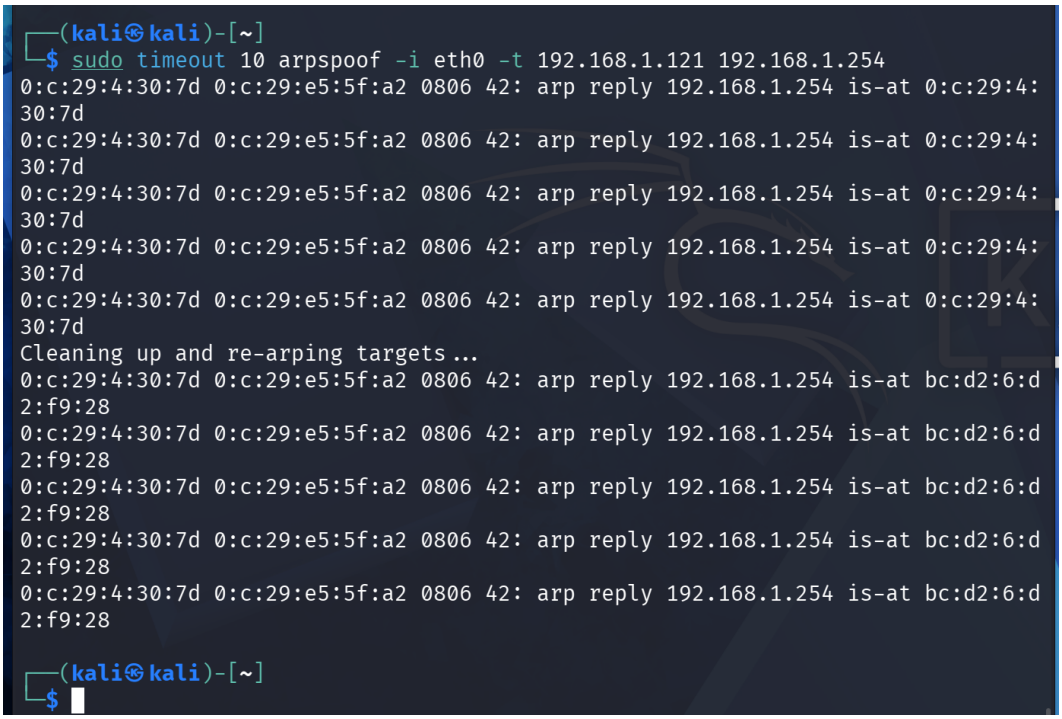
Evidencia P2_E2b — Tabla ARP de Kali tras el arping -U



P2_E2b — arp -a en Kali tras el ARP gratuito: sin cambios en su propia tabla (comportamiento esperado del emisor).

Para observar el efecto real del ARP gratuito sobre la tabla de otro host — que es el mecanismo exacto detrás del ARP Spoofing — se llevó el ejercicio un paso más allá utilizando arpspoof (paquete dsniff), que internamente envía ese mismo tipo de respuestas ARP falsificadas de forma repetida, dirigido a que Metasploitable2 (192.168.1.121) asociara la dirección MAC de Kali con la IP del gateway real (192.168.1.254). Con el reenvío de paquetes IP habilitado previamente (sysctl net.ipv4.ip_forward=1) para no interrumpir la conectividad de la víctima, el ataque se ejecutó repitiendo el envío de ARP replies falsificados de forma continua, condición necesaria dado que las entradas ARP legítimas se refrescan y expiran constantemente.

Evidencia P2_E2c — Ataque de ARP Spoofing completo con efecto verificado en otro host



P2_E2c — Ejecución de arpspoof; nótese la limpieza automática al finalizar.

Consultando la tabla ARP directamente en Metasploitable2 (arp -a) mientras el ataque se mantenía activo en segundo plano, se confirmó que la entrada correspondiente al gateway (192.168.1.254) fue sustituida: en condiciones normales corresponde a la MAC bc:d2:06:d2:f9:28 (el router real), pero durante el ataque aparece asociada a 00:0c:29:04:30:7d (la MAC de Kali). Esto demuestra de forma concluyente que la víctima fue engañada para enviar, a partir de ese momento, todo el tráfico destinado a internet a través del equipo atacante — la condición exacta que habilita un ataque Adversary-in-the-Middle (T1557 de MITRE ATT&CK).

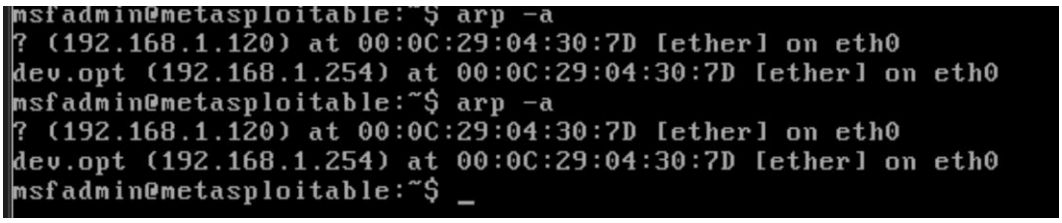


Tabla ARP de Metasploitable2 DURANTE el ataque: gateway suplantado por la MAC de Kali (antes/después vs. condición normal).

Al detener el proceso de arpspoof, la herramienta ejecutó automáticamente su rutina de limpieza, reenviando los ARP replies correctos para restaurar la asociación legítima entre el gateway y su MAC real. Se confirmó la restauración consultando nuevamente la tabla ARP de la víctima.

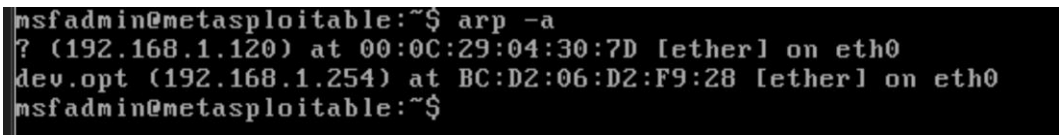
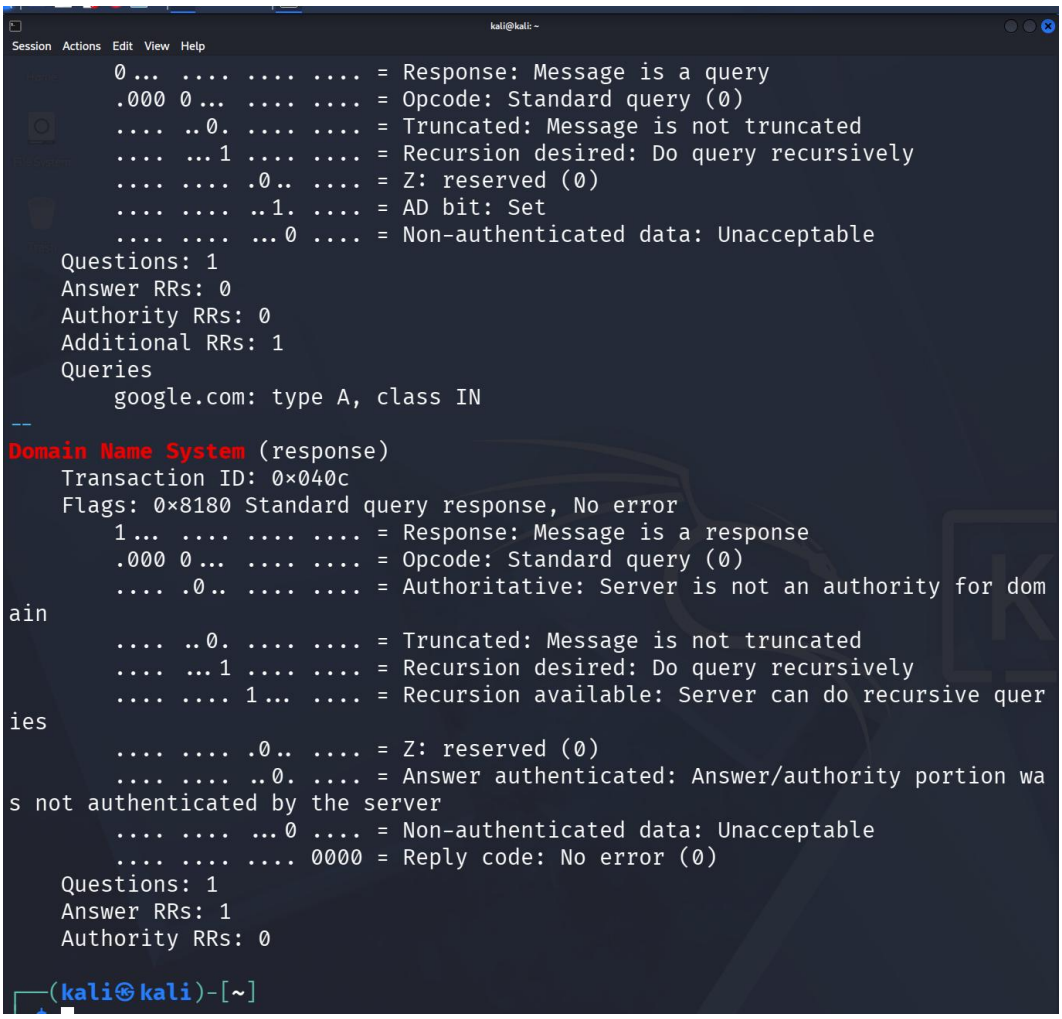


Tabla ARP de Metasploitable2 restaurada tras finalizar el ataque.

Fase 2C — Análisis de Tráfico DNS

Al intentar capturar una consulta DNS con dig, se detectó que las peticiones contra el resolver configurado en Kali (192.168.1.254, el gateway doméstico) no devolvían respuesta, aunque la conectividad IP hacia internet funcionaba con normalidad. Forzando la consulta directamente contra un resolver público (dig +short google.com @8.8.8.8) se confirmó que Kali sí podía resolver nombres correctamente — posiblemente el gateway dejó de responder consultas DNS de este equipo como reacción al tráfico de ARP Spoofing generado en la fase anterior. Con la resolución funcionando, se capturó el intercambio completo de consulta y respuesta DNS con tshark (filtro udp port 53).

Evidencia P2_E3 — Consulta y respuesta DNS con Transaction ID visible



P2_E3 — Consulta y respuesta DNS (google.com, type A) — Transaction ID: 0x040c.

El desglose del paquete confirma la pregunta (Queries: google.com, type A, class IN) y la respuesta exitosa del resolver (Transaction ID: 0x040c; Flags: Standard query response, No error; Answer RRs: 1, con la IP devuelta 142.251.45.14), incluyendo la bandera Recursion

available que confirma que 8.8.8.8 sí opera como resolver recursivo, a diferencia del gateway local que no respondió.

Fase 2D — El Handshake TCP de 3 Vías

Se capturó el tráfico TCP hacia el puerto 80 de Metasploitable2 mientras se generaba una petición HTTP con curl, y se filtraron específicamente los paquetes de establecimiento de conexión. La secuencia capturada muestra el patrón esperado: SYN (Kali → Metasploitable2, Seq=0), SYN-ACK (Metasploitable2 → Kali, Seq=0, Ack=1), y ACK final (Kali → Metasploitable2, Seq=1, Ack=1), completando la negociación en menos de 2 milisegundos por tratarse de tráfico dentro de la misma red local. En un SYN Flood, el servidor jamás recibe el paquete #3 (ACK final) que completaría la conexión, quedando la conexión en estado "half-open" y llenando su tabla de conexiones pendientes (backlog) hasta agotar los recursos disponibles.

Evidencia P2_E4 — Handshake TCP de 3 vías con números de secuencia (0.4 pts, rúbrica ítem 4)

```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -r /tmp/tcp_handshake.pcap -Y "tcp.flags.syn==1 or (tcp.flags.s
yn==0 and tcp.flags.ack==1 and tcp.len==0)" -c 3
Running as user "root" and group "root". This could be dangerous.
    1 0.000000000 192.168.1.120 → 192.168.1.121 TCP 74 53696 → 80 [SYN] Seq=0 W
in=64240 Len=0 MSS=1460 SACK_PERM TSval=4255150366 TSecr=0 WS=1024
    2 0.001658986 192.168.1.121 → 192.168.1.120 TCP 74 80 → 53696 [SYN, ACK] Se
q=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM TSval=843298 TSecr=4255150366 WS=32
    3 0.001818744 192.168.1.120 → 192.168.1.121 TCP 66 53696 → 80 [ACK] Seq=1 A
ck=1 Win=64512 Len=0 TSval=4255150368 TSecr=843298
```

P2_E4 — Handshake TCP de 3 vías (SYN Seq=0, SYN-ACK Seq=0/Ack=1, ACK Seq=1/Ack=1).

Reflexión Requerida — Práctica 2

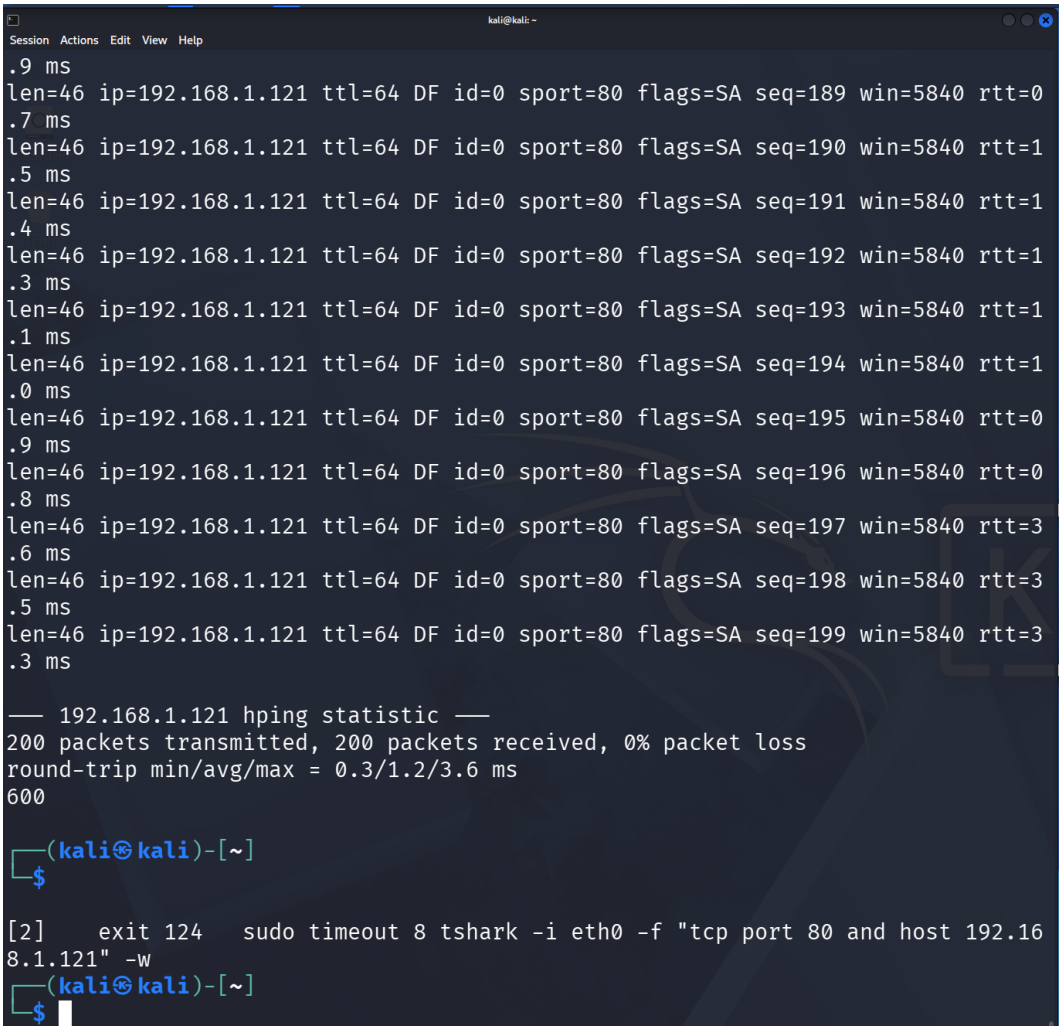
Evidencia P2_E5 — Reflexión escrita

¿Por qué ARP es tan vulnerable? ¿Qué mecanismo hace que el envenenamiento de caché ARP sea posible?

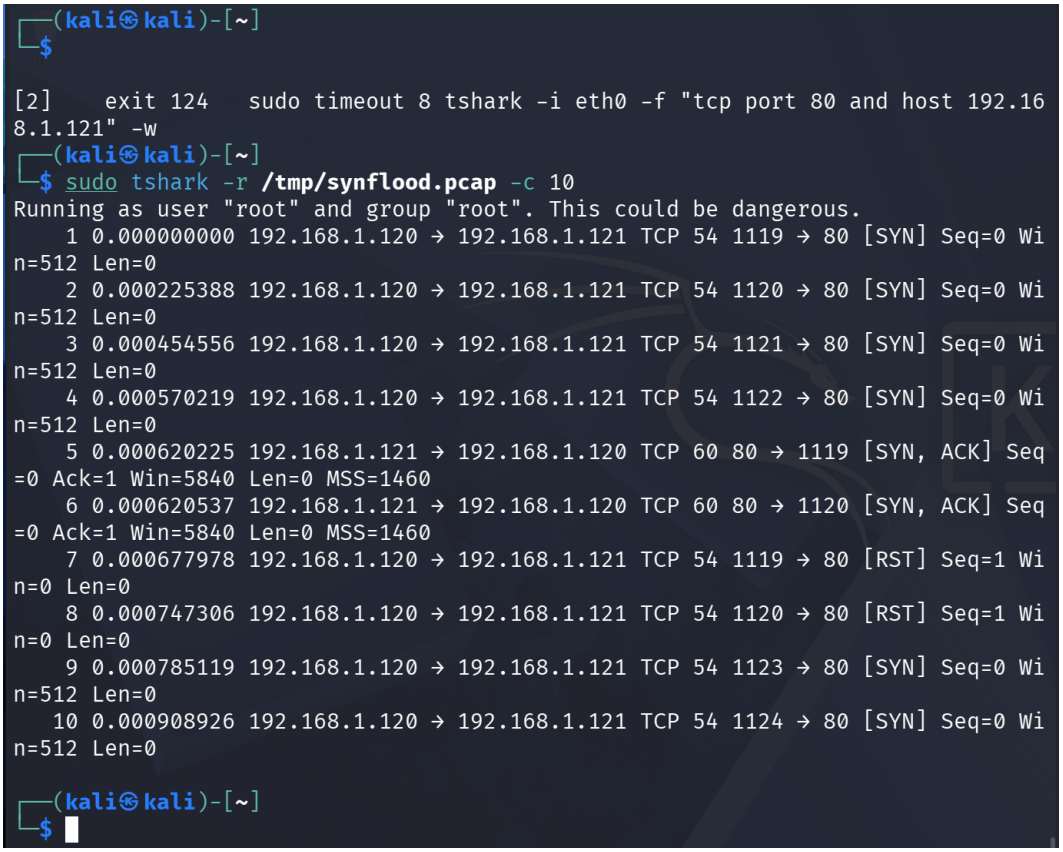
ARP es vulnerable porque el protocolo, diseñado en los años 80, asume que todos los nodos de la red local son confiables y no incluye ningún mecanismo de autenticación: cualquier host puede enviar una respuesta ARP (incluso sin que se le haya preguntado, mediante un "ARP gratuito") y los demás equipos la aceptan sin verificar si proviene realmente del dueño legítimo de esa IP. En mi prueba con arpspoof comprobé esto directamente: al enviar de forma repetida respuestas ARP falsificadas, Metasploitable2 actualizó su tabla ARP asociando la IP del gateway real con la MAC de mi máquina Kali, sin ninguna validación. Esto funciona porque las entradas ARP caducan y se refrescan constantemente, así que basta con enviar respuestas falsas de forma continua para mantener el envenenamiento. El resultado es que la víctima envía todo su tráfico de salida al atacante, habilitando un ataque Adversary-in-the-Middle, sin importar si el contenido posterior va cifrado o no.

Material Adicional (Bonus)

Como evidencia adicional se generó un SYN flood controlado contra el puerto 80 de Metasploitable2 usando hping3 (-S, -c 200, --faster), capturando simultáneamente el tráfico con tshark. El resultado muestra 200 paquetes transmitidos y 200 recibidos (0% de pérdida), con Metasploitable2 respondiendo SYN-ACK a cada intento sin degradación visible — evidencia de que un volumen de 200 paquetes no es suficiente para agotar la tabla de conexiones half-open de un servidor real, a diferencia de un ataque real que además normalmente usa IP origen falsificada.



Bonus — Resultado de hping3: 200 paquetes SYN, 0% de pérdida.



Bonus — Detalle de paquetes: SYN en ráfaga, SYN-ACK del servidor, y RST automático de Kali.

Práctica 3 — Criptografía en Acción: Hash, Cifrado y TLS

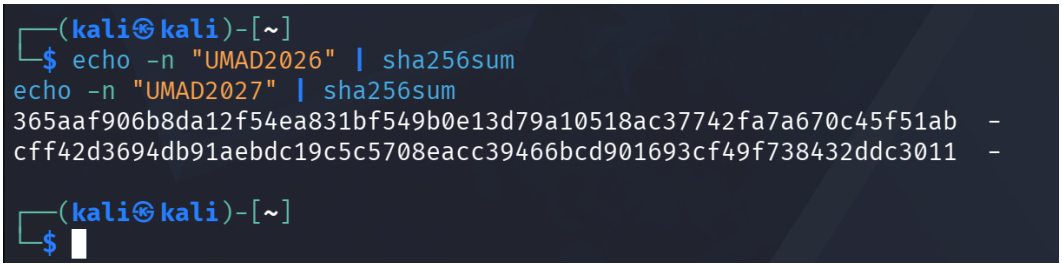
Fase 3A — Funciones Hash (MD5, SHA-256) y Efecto Avalancha

En lugar del texto de ejemplo sugerido en la guía, se utilizó un par de cadenas propias que difieren en un solo carácter (UMAD2026 y UMAD2027, equivalente al cambio de mayúscula/minúscula sugerido) para demostrar el mismo principio: el efecto avalancha. Se calculó el hash de ambas cadenas con MD5 y SHA-256.

Evidencia P3_E1 — Hashes MD5 y SHA-256, original vs. modificado



P3_E1a — MD5 de "UMAD2026" (646b770a...) vs. "UMAD2027" (7e205df8...): salidas totalmente distintas.



P3_E1b — SHA-256 de las mismas cadenas: mismo efecto avalancha, salida de 64 caracteres (256 bits).

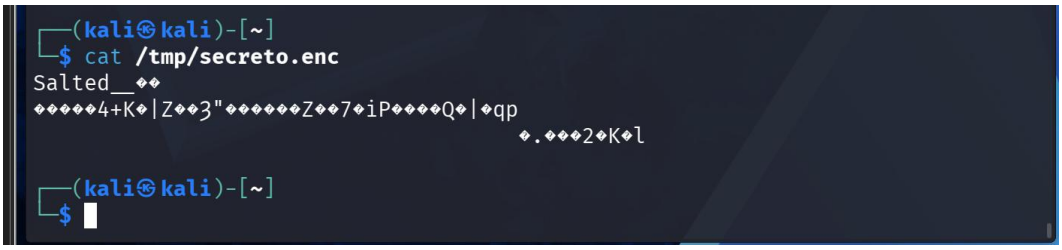
Un cambio mínimo en la entrada (un solo dígito) produjo salidas completamente distintas e impredecibles en ambos algoritmos, sin ningún patrón visible que las relacionara — propiedad esencial para que un hash sirva como verificador de integridad. La diferencia de longitud entre algoritmos (128 bits en MD5 contra 256 en SHA-256) explica en parte por qué MD5 se considera criptográficamente roto para propósitos de seguridad (existen colisiones documentadas desde 2004), mientras que SHA-256 sigue siendo estándar vigente.

Fase 3B — Cifrado Simétrico con AES (OpenSSL)

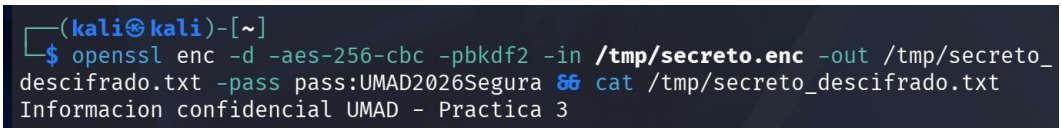
Se creó un archivo de texto plano simulando información confidencial, y se cifró con AES-256 en modo CBC usando OpenSSL, derivando la llave real a partir de una contraseña mediante PBKDF2 (con iteraciones) y agregando un salt aleatorio. Al inspeccionar el archivo resultante

con cat se confirmó que es binario ilegible, iniciando con el encabezado estándar "Salted__" que OpenSSL antepone a todo archivo cifrado con la opción -salt.

Evidencia P3_E2 — Archivo legible vs. cifrado, con cat/xxd



P3_E2a — Contenido del archivo cifrado con AES-256 (cat secreto.enc): binario ilegible, encabezado "Salted__".

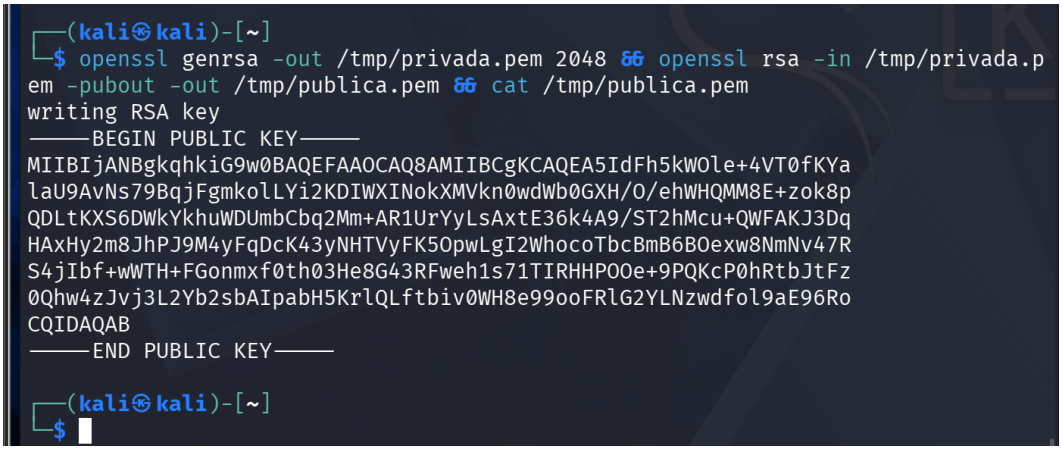


P3_E2b — Descifrado exitoso: el texto original se recupera intacto con la misma contraseña.

Fase 3C — Cifrado Asimétrico RSA y HMAC

Se generó un par de llaves RSA de 2048 bits (privada y pública derivada de esta), demostrando el principio de la criptografía asimétrica: la llave pública puede compartirse libremente sin comprometer la seguridad del sistema, ya que es matemáticamente inviable derivar la llave privada a partir de ella con la tecnología actual. Se cifró un mensaje utilizando exclusivamente la llave pública y se descifró exitosamente sólo con la llave privada correspondiente.

Evidencia P3_E3 — Generación de par RSA y cifrado/descifrado exitoso



P3_E3a — Par de llaves RSA-2048 generado; llave pública en formato PEM.



P3_E3b — Ciclo completo RSA: cifrado con llave pública, descifrado exitoso con llave privada.
Como evidencia adicional (no solicitada explícitamente por la guía, pero relacionada con la protección contra Man-in-the-Middle) se calculó el HMAC-SHA256 del mismo mensaje y de una versión alterada, usando una llave secreta compartida, confirmando que sin conocer la llave secreta un atacante no podría generar un HMAC válido para un mensaje falsificado.

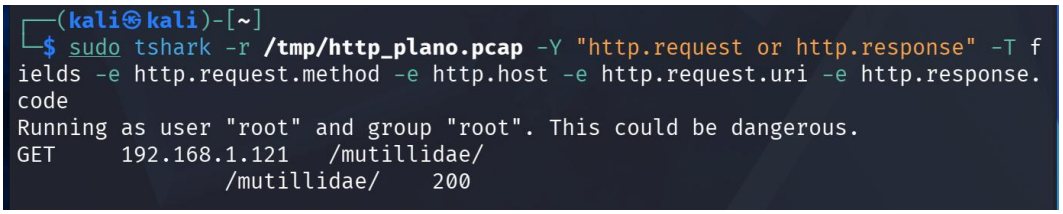


Bonus — HMAC-SHA256 del mensaje original y de una versión modificada: salidas totalmente distintas.

Fase 3D — TLS en Acción con Wireshark/tshark

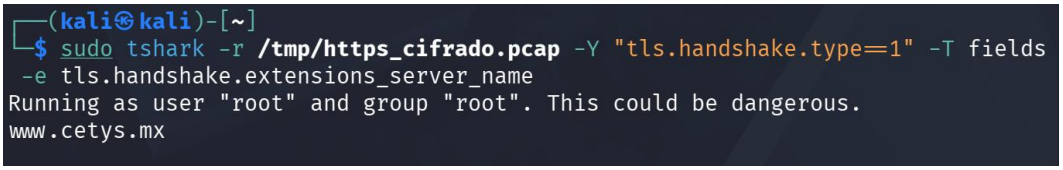
Se capturó una petición HTTP en texto plano contra el servicio web de Metasploitable2 (mutillidae), extrayendo con tshark el método, host, URI y código de respuesta directamente del tráfico capturado, sin ningún esfuerzo de descifrado — todo el contenido de la petición es legible a simple vista.

Evidencia P3_E4 — 2 capturas: HTTP legible vs. HTTPS/TLS cifrado



P3_E4a — Petición HTTP capturada en texto plano: método, host, ruta y código de respuesta visibles.

Como contraste, se capturó tráfico HTTPS hacia un sitio externo real (www.cetys.mx) y se intentó extraer la misma información del handshake TLS. Únicamente el nombre de dominio quedó visible, a través del campo SNI (Server Name Indication) del mensaje Client Hello — una limitación conocida de TLS clásico. Todo lo demás (ruta solicitada, cuerpo de la respuesta, código de estado) permaneció completamente cifrado e inextraíble. Inspeccionando el campo tls.handshake.version del paquete Server Hello (tshark -Y "tls.handshake.type==2") se identificó el valor 0x0303, correspondiente a TLS 1.2 como versión negociada en esta conexión.



P3_E4b — Único dato visible en el tráfico HTTPS: el dominio de destino vía el campo SNI del handshake TLS.

```
┌──(kali㉿kali)-[~]
└─$ sudo tshark -r /tmp/https_cifrado.pcap -Y "tls.handshake.type==2" -T fields
 -e tls.handshake.version
Running as user "root" and group "root". This could be dangerous.
0×0303
```

P3_E4c — Versión de TLS negociada extraída del Server Hello: 0x0303 (TLS 1.2).

Elemento observado	HTTP (puerto 80)	HTTPS (puerto 443)
Método/verbo	GET visible	Cifrado, no extraíble
Host/dominio	192.168.1.121 visible	www.cetys.mx visible (vía SNI)
Ruta/URI	/mutillidae/ visible	Cifrada, no extraíble
Código de respuesta	200 visible	Cifrado, no extraíble
Contenido (body/HTML)	Totalmente visible	Totalmente cifrado

Tabla comparativa — datos observables en tráfico HTTP vs HTTPS capturados con tshark.

Reflexión Requerida — Práctica 3

Evidencia P3_E5 — Reflexión escrita

Explica con tus palabras la diferencia entre cifrado simétrico (AES) y asimétrico (RSA).

¿Cuándo usarías cada uno?

El cifrado simétrico (AES) usa la misma llave para cifrar y descifrar, lo que lo hace muy rápido y eficiente incluso con archivos grandes, pero requiere que ambas partes ya compartan esa llave secreta de forma previa y segura — si alguien la intercepta, puede leer todo lo cifrado con ella. Comprobé esto cifrando un archivo con AES-256 usando una contraseña: el resultado fue un binario completamente ilegible, y solo pude recuperarlo usando exactamente la misma contraseña. El cifrado asimétrico (RSA), en cambio, usa un par de llaves matemáticamente relacionadas pero distintas: lo que se cifra con la llave pública solo puede descifrarse con la llave privada correspondiente. Esto resuelve el problema de distribución de llaves de AES, porque la llave pública puede compartirse abiertamente sin riesgo, pero es mucho más lento computacionalmente, por lo que no es práctico para cifrar grandes volúmenes de datos. En la práctica, ambos se combinan: usaría RSA (o un intercambio de llaves basado en curvas elípticas, como en TLS moderno) únicamente para acordar de forma segura una llave simétrica temporal entre dos partes que no se conocen previamente, y después usaría AES para cifrar el resto de la comunicación o los archivos, aprovechando su velocidad. Esto es exactamente lo que ocurre en cada conexión HTTPS: el handshake TLS usa criptografía asimétrica para acordar una llave de sesión, y el tráfico real se cifra con un algoritmo simétrico como AES.

Conclusiones

A lo largo de este laboratorio se ejecutaron varias técnicas que, en un contexto real sin autorización, constituirían delitos graves conforme a la legislación mexicana vigente. El reconocimiento de red con nmap, el ARP spoofing, la captura de tráfico con tshark, y el SYN flood con hping3 caen dentro de los supuestos del artículo 211 bis del Código Penal Federal, que sanciona el acceso, uso, copia o modificación de información en sistemas o equipos informáticos sin autorización, con penas que pueden ir de meses hasta varios años de prisión dependiendo de si el sistema afectado pertenece a particulares, al Estado, o al sistema financiero (siendo este último el más severamente penado).
Lo que hace legal y ético este ejercicio no es la técnica en sí —las mismas herramientas utilizadas (nmap, arpspoof, tshark, hping3, openssl) son de uso completamente libre y legítimo en la industria de ciberseguridad— sino el contexto de autorización: ambas máquinas (Kali y Metasploitable2) son de mi propiedad, corren en un entorno virtualizado aislado bajo mi control total, y el propósito declarado es exclusivamente educativo dentro de un programa académico formal. Esta distinción de autorización y consentimiento es precisamente el eje central de cualquier marco de pentesting ético: sin un acuerdo explícito de alcance firmado por el propietario del sistema, cualquiera de estas actividades es indistinguible legalmente de un ataque real.

Si una organización operara sin políticas de seguridad ni controles mínimos —por ejemplo, sin segmentación de red, sin monitoreo de tráfico ARP anómalo, con servicios expuestos como vsftpd 2.3.4 (vulnerable a CVE-2011-2523, como el encontrado en Metasploitable2)— el riesgo no sería solo técnico sino legal y financiero para la propia organización. Bajo la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP), cualquier empresa que maneje datos personales y sufra una brecha de seguridad por negligencia en sus controles puede enfrentar sanciones del INAI, independientemente de que el atacante también cometa un delito. A nivel internacional, el Convenio de Budapest sobre Ciberdelincuencia (que México aún no ha ratificado, a diferencia de la mayoría de los países de la OCDE) y la más reciente Convención de las Naciones Unidas contra la Ciberdelincuencia (2024) buscan armonizar precisamente este tipo de responsabilidades entre países, dado que ataques como el ARP spoofing o el DDoS no respetan fronteras jurisdiccionales.
En resumen, esta práctica deja claro que la ciberseguridad ofensiva y la legalidad no son conceptos opuestos, sino que dependen enteramente del marco de autorización en el que se ejecutan: la misma línea de código que en un laboratorio enseña un concepto, ejecutada sin consentimiento contra un tercero, es un delito federal.

Referencias (APA 7)

Cámara de Diputados del H. Congreso de la Unión. (2024). Código Penal Federal. https://www.diputados.gob.mx/LeyesBiblio/pdf/CPF.pdf

Cámara de Diputados del H. Congreso de la Unión. (2010). Ley Federal de Protección de Datos Personales en Posesión de los Particulares. https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf

Cloudflare. (s. f.). ¿Qué es un ataque DDoS? Cloudflare Learning Center. https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/

Consejo de Europa. (2001). Convenio sobre la Ciberdelincuencia (Convenio de Budapest). https://rm.coe.int/1680081561

MITRE. (s. f.). T1040: Network sniffing. MITRE ATT&CK. https://attack.mitre.org/techniques/T1040/

Naciones Unidas. (2024). Convención de las Naciones Unidas contra la Ciberdelincuencia. Oficina de las Naciones Unidas contra la Droga y el Delito. https://www.unodc.org/

Hoja de Respuestas

Nombre: ElVigilante
Fecha de la sesión: 12–13 de septiembre de 2026
Hora inicio: 9 am
Hora fin: 3 pm
IP de VM1 (Kali): 192.168.1.120
IP de VM2 (Víctima): 192.168.1.121

Práctica 1 — Respuestas

¿Cuántos hosts encontró nmap -sn en tu subred?: 11

Puertos abiertos encontrados en VM2 (3 de los 23 detectados):

Puerto 21: Servicio vsftpd 2.3.4 (CVE-2011-2523)

Puerto 6667: Servicio UnrealIRCd (CVE-2010-2075)

Puerto 1524: Servicio Metasploitable root shell (sin autenticación)

¿Cuántos paquetes SYN sin respuesta contaste en Wireshark/tshark?: Al menos 16 visibles en la captura P1_E3 (el filtro permaneció activo durante todo el escaneo -A -T4, por lo que el total real pudo ser mayor; no se contó con precisión exacta por el scroll limitado de la terminal).

Reflexión: un ataque de reconocimiento recolecta información sobre el objetivo sin explotar nada todavía. Nmap es legal en mi propia red porque soy el dueño y administrador de los equipos escaneados; el mismo comando contra una red ajena sin autorización constituye acceso ilícito bajo el artículo 211 bis del CPF (ver reflexión completa en la sección de Práctica 1).

Práctica 2 — Respuestas

MAC de VM2 según arp -a: 00:0c:29:e5:5f:a2

¿Qué sucedió con la tabla ARP después del arping -U?: Sin cambios en la tabla de Kali (comportamiento correcto: el emisor no actualiza su propia caché a partir de sus propios anuncios). El efecto real del mecanismo de ARP gratuito sobre la tabla de otro host se verificó con arpspoof: la entrada del gateway (192.168.1.254) en la tabla de Metasploitable2, normalmente asociada a la MAC real del router (bc:d2:06:d2:f9:28), fue sustituida por la MAC de Kali (00:0c:29:04:30:7d) mientras el ataque estuvo activo, y se restauró automáticamente al detener arpspoof.

Transaction ID de la primera consulta DNS: 0x040c

IP devuelta para google.com: 142.251.45.14

Número de secuencia del SYN (paquete 1 del handshake TCP): Seq=0 (relativo, según tshark)

Reflexión: ARP es vulnerable porque no incluye autenticación — cualquier host puede enviar una respuesta ARP (incluso sin haber sido consultado) y los demás equipos la aceptan sin verificarla. Esto, combinado con la caducidad periódica de las entradas ARP, es lo que hace posible el envenenamiento sostenido de la caché (ver reflexión completa de 150 palabras en la sección de Práctica 2).

Práctica 3 — Respuestas

Hash MD5 de la cadena de prueba ("UMAD2026"): 646b770a0ecf27890214dfafe12f5d7f

Hash MD5 de la cadena modificada ("UMAD2027"): 7e205df8725892b58b24be755004e7f6

¿El cifrado AES cambia significativamente el tamaño del archivo?: No se documentó una comparación explícita con ls -lh en esta sesión; conceptualmente, el cifrado por bloques con salt e IV agrega solo unos pocos bytes de overhead, sin un cambio drástico de tamaño.

¿Puedes leer el tráfico HTTP en Wireshark/tshark?: SÍ

¿Puedes leer el tráfico HTTPS/TLS?: NO (excepto el dominio, visible vía el campo SNI)

Versión de TLS negociada en la conexión HTTPS: TLS 1.2 (campo tls.handshake.version = 0x0303 en el Server Hello)

Reflexión: AES (simétrico) usa la misma llave para cifrar y descifrar — rápido pero requiere compartir la llave de antemano. RSA (asimétrico) usa un par de llaves pública/privada — resuelve el problema de distribución de llaves pero es más lento. En la práctica se combinan: RSA/TLS para acordar una llave de sesión, y AES para cifrar el tráfico real (ver reflexión completa de 200 palabras en la sección de Práctica 3).